

THREADS Y PROCESOS

Para comprender mejor el tema de programación concurrente es necesario conocer y dominar dos conceptos claves, me refiero a threads y procesos, conceptos que en primeras instancias puede ser algo complejos de comprender y que, sin duda, a más de uno le han causado dolores de cabeza. Es por ello por lo que en esta ocasión me gustaría que habláramos sobre ellos, explicando qué son, en que son similares y por supuesto, en que son diferentes.

PROCESOS

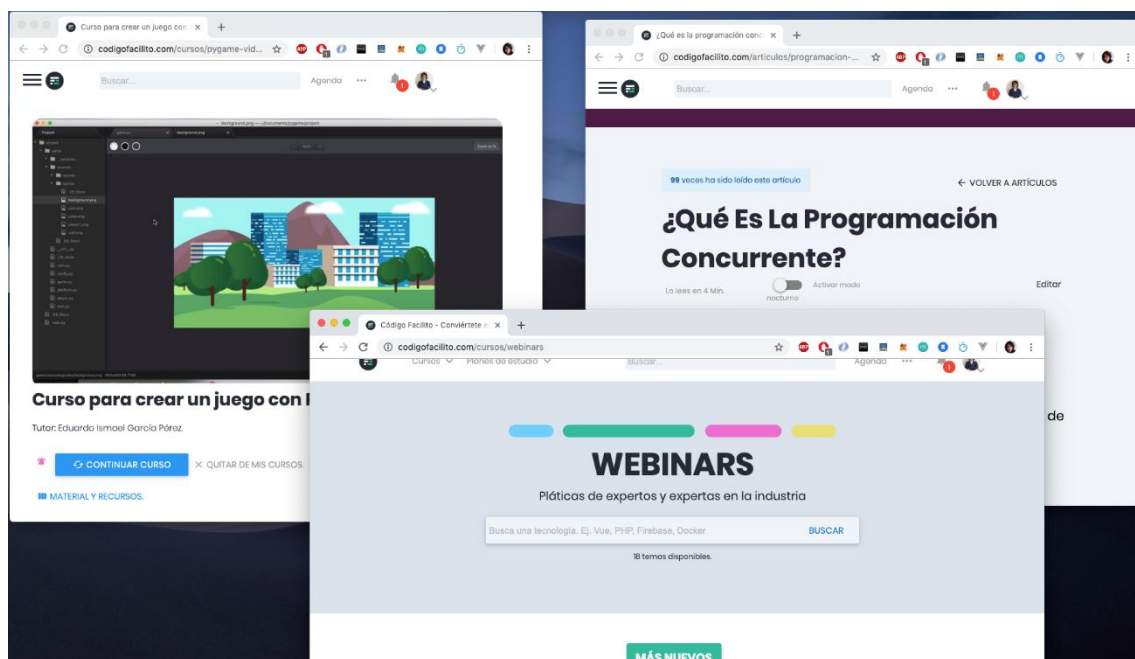
¿Qué es un Proceso?

En términos simples un **proceso** no es más que la **instancia de un programa**. Verás, cuando tú abres cualquier programa en tu dispositivo, por ejemplo, tu navegador web, quizás google chrome, internamente estás creando un nuevo proceso.

Un **proceso** no será más que la **ejecución del programa** mismo. Y ojo, no hay que confundir un proceso con un programa ya que son entidades completamente diferentes. Podemos ver a un **programa** como un **conjunto de instrucciones y de datos**, por otro lado, podemos ver a un **proceso** como la **ejecución de esas instrucciones junto con esos datos**. Una muy buena analogía es ver a un programa como la receta para preparar un platillo, ya que en una receta se establecen todos los ingredientes a utilizar junto con los pasos a seguir; en cambio, un proceso podemos verlo como la ejecución de esa receta, algo que se está realizando.

Puede quedarnos más en claro si consideramos que un mismo programa puede ser ejecutado múltiples veces por nuestro sistema operativo. Siguiendo con la analogía, una misma receta puede ser implementada por n chefs.

En la siguiente imagen podemos observar tres procesos diferentes del mismo programa.



Dentro de un proceso encontraremos todo lo necesario para que el programa se ejecute de forma correcta, me refiero a código fuente, ficheros, variables, tareas, subprocessos etc...

Otra entidad importante para comprender mejor el tema de procesos es sin duda el sistema operativo. El **sistema operativo** será **quien cree, ejecute y controle todos los procesos**.

Una de las tareas del sistema operativo es **aislar a cada uno de los procesos entre sí**, de tal forma que los **procesos** sean **completamente independientes unos de otros**; esto con la finalidad prevenir que compartan información entre ellos, lo cual pueda dar pie a errores. Es por ello que, en esencia, los programas no pueden acceder a la información y datos de otros programas en ejecución.

Cuando un proceso finaliza, ya sea de forma natural o no, quizás por algún error, será el sistema operativo el encargado de liberar el espacio en memoria.

Podemos ver al sistema operativo como un maestro de ceremonias, quien está al pendiente de todos los procesos, desde su creación, ejecución y finalización.

Si utilizas un sistema operativo basado en unix, por ejemplo, fedora, puedes ejecutar en tu terminal el comando top, el cual listará todos los procesos en ejecución de tu sistema, de esta forma podrás obtener información de ellos.

Con los procesos es posible implementar el **paralelismo**, ejecutando diferentes procesos en diferentes procesadores, claro, también depende de cuantos procesadores posea el equipo de cómputo.

Para que **un proceso** pueda ser ejecutado este **necesita poseer por lo menos un thread**, a este thread lo vamos a conocer como el **thread principal, o, el main thread**.

THREADS

Los **Threads**, también conocidos como **sub-procesos o hilos**, podemos definirlos como una **secuencia de instrucciones las cuales el sistema operativo puede programar para su ejecución**. A diferencia de un proceso, los threads son entidades mucho más pequeñas, lo cual los hace fáciles de gestionar, tanto es así **que un thread es la unidad más pequeña a la cual un procesador puede asignar tiempo**.

A diferencia de los procesos, los cuales viven dentro del sistema operativo, los threads viven dentro de los procesos. Un thread se crea, ejecuta y finaliza dentro de un proceso, dicho en otras palabras: **Un thread le pertenece a un proceso**, y, a su vez, un proceso puede poseer múltiples threads. Si lo vemos en términos de base de datos podríamos decir que esta es una relación uno a muchos.

Algo interesante a mencionar es que debido a que los Thread existen dentro de los procesos, estos **pueden compartir información entre ellos**; Algo a lo cual sin duda le podemos sacar mucho provecho, pero, que también puede llegar a ser complicado de manejar. Uno de los problemas más comunes al momento de trabajar con Threads tiene por nombre Race condition, y es, en esencia, un **problema** que surge **cuando más de un thread intenta acceder y modificar un espacio en memoria compartido**, ocasionando que el programa se comporte de forma inadecuada.

Visto de otra forma, un thread no es más que un proceso, pero en pequeño, al igual de un proceso un thread se ejecuta dentro de un contexto y posee instrucciones a realizar,

CONCLUSIÓN

En conclusión, un **proceso** no es más que **la instancia de un programa, el cual es creado y controlado por el sistema operativo**. Los procesos **son entidades independientes y no pueden compartir información entre ellos**. Para que un proceso pueda ser ejecutado es necesario que este **posea por lo menos un thread**.

Por otro lado, un **thread** es la **unidad más pequeña a la cual un procesador puede asignar tiempo**. Los threads poseerán la **secuencia más pequeña de instrucciones a ejecutar**. Los threads se **crean, ejecutan y mueren dentro de los procesos, siendo capaces de compartir información entre ellos**.

Con los threads y los procesos seremos capaces de implementar la **programación concurrente**, y, dependiendo de la cantidad de procesadores la programación en paralelo.